CSS311 - Parallel & Distributed Computing

PROGRAMMING ASSIGNMENT - 2



Field	Value
Group Number	Group 22
Group Leader	2024BCS0046 - Noel Manoj
Group Member 2	2024BCS0038 - Aditya A
Group Member 3	2024BCD0018 - Adwaith J
Semester & Batch Number	Sem V & Batch 3
Application Theme	Image Processing
Assigned Problem / Title	Image Sharpening using 2D Convolution
Submission Date	07/10/2026
Drive Link	Click Here

Part A - MPI Implementation (Version 5 and Version 6)

A.1 Problem Recap

This project implements **2D spatial convolution** using a standard 3×3 Laplacian sharpening kernel on raw 8-bit grayscale images. The same problem, input format, datasets (Small: 1920×1080, Medium: 3840×2160, Large: 7680×4320), and correctness checks from PA1 are used across all six implementations. The sharpening kernel is:

```
 0  -1   0
-1   5  -1
 0  -1   0
```

Each output pixel is computed as a weighted sum of its 3×3 neighborhood, with zero-padding at boundaries and clamping to [0, 255].

A.2 Version 5 - Basic MPI Implementation

Domain Decomposition

The image is decomposed using **1D block decomposition along the row (height) dimension**. Each MPI rank receives a contiguous block of `H / size` rows (with the last rank absorbing any remainder rows when the height is

not evenly divisible by the process count).

- **Type:** Block decomposition (contiguous row chunks)
- **Why block (not cyclic or block-cyclic):** Since every row in a 2D convolution has identical computational cost (uniform workload), block decomposition provides perfect load balance without the additional communication complexity that cyclic or block-cyclic distributions would introduce. Block distribution also maximizes spatial locality - each rank operates on a contiguous memory region, which is cache-friendly.

Communication Points

Communication Operation	What Is Exchanged	Between Which Ranks	Frequency
`MPI_Scatterv`	Input image rows (contiguous blocks)	Rank 0 → All ranks	Once (before computation)
`MPI_Gatherv`	Output image rows (processed blocks)	All ranks → Rank 0	Once (after computation)
Halo row exchange (`MPI_Sendrecv`)	1-pixel-wide boundary rows (ghost rows)	Adjacent ranks (rank i ↔ rank $i \pm 1$)	Once (before computation)

- **Halo Exchange Detail:** Because the 3×3 convolution kernel requires access to the row above and below each pixel, each rank needs one "ghost row" from its upper neighbor and one from its lower neighbor. This is implemented via `MPI_Sendrecv` between adjacent ranks before the convolution begins.
- **Total Communication Volume:** For an image of width `W` and `P` processes, each halo exchange transfers `2 × W` bytes (one row up, one row down) per interior rank boundary. Total halo data = `2 × (P-1) × W` bytes.

Implementation Details

```
// Core computation: each rank processes its local rows (including halo rows)
void sharpen_local(const unsigned char* local_input, unsigned char* local_output,
                   int local_rows, int width, int has_top_halo, int has_bottom_halo) {
    int start_row = has_top_halo ? 1 : 0;
    int end_row = has_bottom_halo ? local_rows - 1 : local_rows;
    for (int r = start_row; r < end_row; ++r) {
        for (int c = 0; c < width; ++c) {
            int sum = 5 * local_input[r * width + c]
                    - local_input[(r-1) * width + c]
                    - local_input[(r+1) * width + c]
                    - local_input[r * width + (c-1)]
                    - local_input[r * width + (c+1)];
            // Boundary handling for left/right edges and clamping
            local_output[(r - start_row) * width + c] =
                static_cast<unsigned char>(std::max(0, std::min(255, sum)));
        }
    }
}
```

Correctness Verification

The MPI v5 output was verified against a **sequential golden baseline** generated independently. Per-pixel comparison across all three datasets produced: - **Max Absolute Difference:** 0 - **Mismatched Pixels:** 0 / total pixels - **Result:** PASSED - Output matches Sequential Golden Baseline exactly.

A.3 Version 6 - Optimized MPI Implementation

Version 6 incorporates two synergistic optimizations: 1. **1D Cartesian Virtual Topology (`MPI_Cart_create` & `MPI_Cart_shift`)** for clean, hardware-aware process topology mapping and boundary handling. 2. **Non-Blocking Halo Exchange (`MPI_Irecv` / `MPI_Isend`) with Loop Splitting** to overlap halo communication latency with useful interior row computation.

Optimization 1: 1D Cartesian Virtual Topology

Instead of manually calculating neighbor ranks with arithmetic and branch checks (`rank - 1` and `rank + 1`), Version 6 establishes a **1D Cartesian Virtual Topology**:

- `MPI_Cart_create(MPI_COMM_WORLD, 1, dims, periods, 0, &cart_comm)` :
- Configures a 1-dimensional process grid (`dims[0] = world_size`).
- Sets `periods[0] = 0` (non-periodic, non-wrapping) since image boundaries terminate at row 0 and row H - 1.
- Allows the underlying MPI runtime to optimize process-to-core mapping based on physical processor geometry.
- `MPI_Cart_shift(cart_comm, 0, 1, &prev_rank, &next_rank)` :
- Automatically determines neighbor ranks (`prev_rank` and `next_rank`).
- For edge processes (Rank 0 top neighbor, Rank P - 1 bottom neighbor), `MPI_Cart_shift` automatically returns `MPI_PROC_NULL` .
- **Branch-free execution:** Because MPI primitives (`MPI_Irecv` , `MPI_Isend`) silently treat `MPI_PROC_NULL` as a valid no-op, ranks can post halo communications uniformly without branching (`if (rank > 0) ...`).

Optimization 2: Non-Blocking Halo Exchange with Computation-Communication Overlap

The 2D stencil convolution has a natural separation between: 1. **Interior rows** (rows 1 to `local_rows - 2`): These rows depend exclusively on pixels already residing in local rank memory - they require zero halo data. 2. **Boundary rows** (row 0 and row `local_rows - 1`): These rows depend on ghost halo rows exchanged with top and bottom neighbors.

By issuing non-blocking halo exchanges and immediately computing interior rows while the halo data is in transit, we can **overlap communication latency with computation**:

```
Timeline (Version 5 - Blocking):
  [Halo Exchange] ———————— [Compute ALL rows]

Timeline (Version 6 - Non-Blocking + Cartesian):
  [MPI_Cart_shift Neighbor Lookup]
  [Post MPI_Irecv & MPI_Isend]
  [Compute INTERIOR rows]  ← overlaps with halo transit in background
  [MPI_Waitall for Halos]
  [Compute BOUNDARY rows]
```

Implementation Snippet (from `v6_mpi_optimized.cpp`)

```
// 1. Establish 1D Cartesian Topology
int dims[1] = {world_size};
int periods[1] = {0}; // Non-periodic
MPI_Comm cart_comm;
MPI_Cart_create(MPI_COMM_WORLD, 1, dims, periods, 0, &cart_comm);

int rank;
MPI_Comm_rank(cart_comm, &rank);

int prev_rank, next_rank;
MPI_Cart_shift(cart_comm, 0, 1, &prev_rank, &next_rank);

// 2. Post Non-blocking Receives for Ghost/Halo Rows
MPI_Request reqs[4];
int req_count = 0;
MPI_Irecv(&local_buf[0], W, MPI_BYTE, prev_rank, 100, cart_comm, &reqs[req_count++]);
MPI_Irecv(&local_buf[(local_rows + 1) * W], W, MPI_BYTE, next_rank, 101, cart_comm, &reqs[req_count++]);

// 3. Post Non-blocking Sends for Local Boundary Rows
MPI_Isend(&local_buf[1 * W], W, MPI_BYTE, prev_rank, 101, cart_comm, &reqs[req_count++]);
MPI_Isend(&local_buf[local_rows * W], W, MPI_BYTE, next_rank, 100, cart_comm, &reqs[req_count++]);

// 4. Overlapped Compute: Process Interior Rows (No Halo Dependency)
for (int r = 2; r < local_rows; ++r) {
    for (int c = 0; c < W; ++c) {
        // Compute 3x3 convolution for interior pixels
        ...
    }
}

// 5. Synchronize Halo Communication
MPI_Waitall(req_count, reqs, MPI_STATUSES_IGNORE);

// 6. Compute Boundary Rows (Row 1 and Row local_rows, now having fresh halos)
process_boundary_row(1);
process_boundary_row(local_rows);
```

Correctness Verification

Same golden baseline verification as v5: - **Max Absolute Difference:** 0 - **Mismatched Pixels:** 0 / total pixels - **Result:** PASSED - Output matches Sequential Golden Baseline exactly.

A.4 MPI Benchmark Results

All benchmarks were executed on a **Windows host using MS-MPI**, with 3 runs per configuration and averaged. Process counts tested: **2, 4, and 8 ranks**.

Version 5 (Basic MPI) - Execution Times

Dataset	Resolution	2 Ranks (ms)	4 Ranks (ms)	8 Ranks (ms)
Small	1920×1080	6.235	5.805	8.258
Medium	3840×2160	22.759	14.430	16.175
Large	7680×4320	91.915	57.990	57.249

Version 6 (Optimized MPI) - Execution Times

Dataset	Resolution	2 Ranks (ms)	4 Ranks (ms)	8 Ranks (ms)
Small	1920×1080	5.322	6.065	8.791
Medium	3840×2160	27.208	16.432	18.149
Large	7680×4320	104.906	61.384	69.384

V5 vs V6 Comparative Speedup

Dataset	Ranks	V5 Time (ms)	V6 Time (ms)	Speedup (V5/V6)
Small	2	6.235	5.322	**1.172**
Small	4	5.805	6.065	0.957
Small	8	8.258	8.791	0.939
Medium	2	22.759	27.208	0.836
Medium	4	14.430	16.432	0.878
Medium	8	16.175	18.149	0.891
Large	2	91.915	104.906	0.876
Large	4	57.990	61.384	0.945
Large	8	57.249	69.384	0.825

A.5 Communication Bottleneck Analysis

Is Communication a Bottleneck?

At small process counts (2 ranks): Communication overhead is minimal. Each rank exchanges only 2 rows of halo data (2 × W bytes). For the Large dataset (W=7680), this is approximately 15 KB - negligible compared to the ~15 MB of local computation data. Communication is **not** a bottleneck.

At large process counts (8 ranks): Communication overhead becomes more significant relative to computation: - Each rank processes fewer rows (~540 rows for Large at 8 ranks), reducing computation time. - The number of halo exchanges increases (7 boundaries for 8 ranks = 14 row transfers). - Process management overhead (spawning, synchronization) is amplified on a single physical machine running via MS-MPI. - The **computation-to-communication ratio decreases**, causing diminishing returns. This aligns with **Amdahl's Law**: as the parallelizable portion shrinks per rank, the fixed communication overhead becomes a larger fraction of total time.

Why V6 Shows Slower Performance Than V5

The optimized version (V6) paradoxically shows **slower** execution than V5 in most configurations. This is attributed to:

1. **Shared-memory MPI (local machine):** On a single machine, MS-MPI uses shared-memory transport for inter-process communication. Halo exchanges complete in sub-microsecond times, leaving essentially zero latency to overlap with computation.
2. **Loop-splitting overhead:** V6 splits the computation loop into three phases (interior, wait, boundary), which prevents the compiler from applying certain loop optimizations (vectorization, prefetching) that are available to V5's single contiguous loop.
3. **Non-blocking API overhead:** `MPI_Irecv` / `MPI_Isend` + `MPI_Waitall` have higher per-call overhead than the simpler `MPI_Sendrecv` used in V5, due to request object management and completion polling.

Conclusion: The non-blocking halo exchange optimization is designed for **distributed-memory clusters** where network latency is 1–100 µs. On a shared-memory local machine, this latency is ~0.01 µs, making the optimization's benefit unmeasurable while its overhead remains constant.

Part B - Performance Analysis Using Tools

B.1 Profiling (CPU and GPU Versions)

B.1.1 CPU Profiling (Sequential, OpenMP, Optimized OpenMP)

Profiling for all CPU implementations was conducted on the Google Colab environment across multiple profiling utilities (`gprof` , `perf record` , and **Valgrind** `cachegrind`) on the Large dataset (7680×4320, 33.18 MB).

> **Methodological Note on Tool Selection & Virtualization:** 1. **Hardware Performance Counters (`perf stat`):** Executing `perf stat -e cycles,instructions,cache-references,cache-misses` in the Google Colab KVM container returned `<not supported>` because virtualized guest containers do not expose underlying hardware PMU registers. 2. **Multi-Threaded `gprof` vs. `perf record` :** Because standard `gprof` relies on POSIX interval timers (`ITIMER_PROF`) that only sample the primary thread (attributing OpenMP parallel worker execution to frame initialization), sampling profiling was augmented with Linux `perf record -e cpu-clock -F 999 -g` , which accurately captures thread call trees across all active worker threads. 3. **Cache Simulation:** Architectural cache hit/miss rates were captured using **Valgrind Cachegrind** with explicit cache hierarchy simulation (`--cache-sim=yes`).

---

1. CPU Hotspot Function Breakdown

Version	Primary Hotspot Symbol	% of Runtime	Secondary Symbol / Overhead	% of Runtime	Profiling Tool Used	Key Technical Findings
Sequential	`sharpen_sequential()`	**100.0%** (gprof) **94.3%** (Cachegrind)	`__memset_avx2` / I/O	**5.7%** (Cachegrind)	`gprof` & Valgrind `cachegrind`	The 3×3 stencil loop with 4 bounds checks per tap consumes almost the entire instruction stream (1.128 Billion instructions).
OpenMP (4T)	`sharpen_omp [clone ._omp_fn.0]`	**80.21%**	Kernel / Thread Runtime (`GOMP`)	**19.79%** (8.97% kernel + 10.82% runtime/IO)	`perf record` (`cpu-clock` @ 999 Hz)	The parallelized convolution loop is the dominant hotspot. Hyperthreading contention on 1 physical core increases thread runtime and kernel synchronization overhead.
Optimized OpenMP (4T)	`sharpen_omp_opt [clone ._omp_fn.0]`	**73.02%**	Kernel / Thread Runtime & Boundary	**26.98%** (12.59% kernel + 14.39% boundary/IO)	`perf record` (`cpu-clock` @ 999 Hz)	Branch elimination and SIMD vectorization reduce compute time to 102.36 ms; boundary processing and memory allocation become proportionately higher.

---

2. Cache Hierarchy & Memory Simulation (Sequential Baseline via Cachegrind)

Simulation on the Large dataset (7680 × 4320) using Valgrind Cachegrind (I_1: 32 KB, D_1: 32 KB, LL: 56 MB, 64-byte lines):

Cache Level	Total References	Cache Misses	Miss Rate	Analysis
Instruction Cache (I_1 / L1i)	1,196,300,643 instructions	2,679 misses	**0.00%**	The instruction footprint of the convolution loop easily fits in L1I cache.
Data Read Cache (D_1 Read)	199,555,254 reads	532,602 misses	**0.27%**	Row-major contiguous reading yields high spatial locality.
Data Write Cache (D_1 Write)	99,718,264 writes	1,557,595 misses	**1.56%**	Sequential write buffering with occasional line invalidations.
Combined Data (D_1)	299,273,518 refs	2,090,197 misses	**0.70%**	Excellent L1 data cache efficiency.
Last-Level Data (LLd)	2,092,876 refs	1,679,193 misses	**0.56%**	Only cold misses to 33.18 MB image buffer reach DRAM.

---

3. Assembly Vectorization Verification (Optimized OpenMP)

Inspection of the generated binary disassembly (`objdump -d -C sharpen_omp_opt`) verified: - **42 AVX/AVX2 Vector Instructions** in the inner loop (`vmovd` , `vpand` , `vpsub` , `vpmul` , `vpadd` , `vpackuswb`). - Zero branching instructions in the interior row compute loop. All 9 pixels of the 3×3 stencil are processed in parallel using 256-bit SIMD registers (`%ymm0` – `%ymm7`), saturating ALU compute pipelines.

B.1.2 GPU Profiling (CUDA)

Profiling for the CUDA implementation was performed on Google Colab using modern NVIDIA profiling utilities (**NVIDIA Nsight Systems** `nsys` and **NVIDIA Nsight Compute** `ncu`).

> **Note on Tool Selection (`ncu` / `nsys` vs `nvprof`):** As verified in the notebook, executing `nvprof` on the modern NVIDIA Tesla T4 (Turing CC 7.5 architecture, CUDA Toolkit 12+) returned `nvprof: command not found` . Starting with the Turing architecture, NVIDIA deprecated `nvprof` and replaced it with **Nsight Systems (`nsys`)** for system-wide tracing and **Nsight Compute (`ncu`)** for in-depth SM-level kernel profiling. Both tools were used to capture comprehensive metrics.

Hardware Platform: - **GPU:** NVIDIA Tesla T4 (Architecture: Turing, Compute Capability 7.5) - **Streaming Multiprocessors (SMs):** 40 SMs (20 TPCs) - **VRAM:** 15,360 MiB GDDR6 (DRAM Frequency: 5.00 GHz) - **Driver Version:** 580.82.07

---

1. System Timeline & Memory Transfer Breakdown (Nsight Systems `nsys`)

Profiling with `nsys profile --stats=true ./cuda_sharpen input_large.bin out_prof.bin 7680 4320` revealed the end-to-end breakdown between memory transfers and kernel compute on the Large dataset:

Operation	Total Time (ns)	Time (ms)	% of Time	Data Transferred	Effective Bandwidth
Host-to-Device Copy (`H2D`)	7,032,125 ns	7.032 ms	48.3%	33.178 MB	4.72 GB/s
Device-to-Host Copy (`D2H`)	7,540,950 ns	7.541 ms	51.7%	33.178 MB	4.40 GB/s
Combined PCIe Transfers	**14,573,075 ns**	**14.573 ms**	**100.0%**	**66.356 MB**	**4.55 GB/s**
Kernel Compute (`sharpen_kernel`)	2,174,817 ns	2.175 ms	-	-	-
End-to-End Total	**16,747,892 ns**	**16.748 ms**	-	-	-

Key Finding: PCIe memory transfers consume **87.0%** of total end-to-end execution time (14.57 ms / 16.75 ms). Compute kernel execution accounts for only **13.0%** of total wall-clock time.

---

2. Standalone Benchmark Results (Across All Datasets)

Measured across 5 repeated runs (from notebook cells 3 and 9):

Dataset	Resolution	Kernel Time (ms)	Total Time (ms)	Transfer Overhead (ms)	Transfer %
Small	1920×1080	0.21 ms	1.42 ms	1.21 ms	85.2%
Medium	3840×2160	0.42 ms	4.35 ms	3.93 ms	90.3%
Large	7680×4320	1.43 – 1.55 ms	15.87 – 18.80 ms	14.44 – 17.37 ms	91.0 – 92.4%

---

3. In-Depth Kernel Profiling (NVIDIA Nsight Compute `ncu`)

Using `ncu --set full ./cuda_sharpen` on the Large dataset (7680×4320, 33,177,600 pixels), detailed microarchitectural metrics were collected for `sharpen_kernel` :

Metric Category	Metric Name	Measured Value	Analysis / Interpretation
Launch Setup	Block Size	`256` (16×16×1)	Standard 2D thread block structure
	Grid Size	`129,600` (480×270×1)	Perfectly tiles 7680×4320 pixels
	Total Threads	`33,177,600`	1 thread per pixel
	Registers / Thread	`22` registers	Low register pressure; allows high occupancy
	Shared Memory / Block	`0 bytes`	Direct global memory access model
Occupancy	Theoretical Occupancy	**100.0%** (32 warps/SM)	Hardware limit on active warps
	Achieved Occupancy	**76.35%** (24.43 warps/SM)	Warp scheduling stalls and register block limits
	Block Limit SM	10 blocks (limited by regs)	22 regs × 256 thd = 5,632 regs/block; max 10 blocks
Speed-of-Light	Compute (SM) Throughput	**49.82%**	Moderate compute pipeline utilization
	DRAM Throughput	**24.92%**	Fraction of peak sustained DRAM bandwidth
	Memory Bandwidth	**79.62 GB/s**	Sustained device memory throughput
	L1/TEX Cache Hit Rate	**75.34%**	Stencil overlap provides high L1 cache hit rate
	L2 Cache Hit Rate	**76.09%**	Intermediate caching reduces DRAM traffic
Instruction Exec.	Executed Instructions	`75,686,400`	Total SASS instructions executed
	Executed IPC	`1.52` inst/cycle	Instructions retired per active cycle
	SM Busy	`38.41%`	Cycles with active compute pipelines
	Branch Efficiency	**100.0%** (0 divergent)	Boundary condition handled uniformly across blocks
Memory Workload	Global Load Data Requested	`298.53 MB`	Total bytes requested by 3×3 stencil reads
	Global Store Data Requested	`33.18 MB`	Exact output image size written
	Load Sector Efficiency	**37.56%** (12.02 / 32 B)	Uncoalesced halo reads cause 61% excessive sectors
	Store Sector Efficiency	**50.00%** (16.00 / 32 B)	Direct byte stores to contiguous memory

4. Primary Bottleneck Identified:

- **Top Warp Stall Cause (34.0% of all warp stalls):** Warps spend 5.4 cycles per instruction stalled waiting for the **L1 instruction queue for local and global (LG) memory operations**. Because each thread performs 9 independent uncoalesced global byte loads without shared-memory tiling, the memory issue queues saturate.
- **Optimization Recommendation:** Implementing a 2D shared-memory tile with a 1-pixel halo would load each tile cooperatively in coalesced 32-bit/64-bit transactions, eliminating 61% of redundant global memory transactions and boosting memory throughput to >80% of peak.

B.2 MPI Profiling (Version 5 & Version 6)

B.2.1 Profiling Methodology & Environment

- **Primary Tool Attempt:** Attempted `mpiP` (lightweight PMPI profiling library).

- *Result:* `mpiP` requires POSIX timers, libunwind, and binutils on Linux/Unix. It is **not available on native Windows with Microsoft MPI (MS-MPI)**.
- **Hardware Fallback Applied (per Section B.4):** High-precision manual profiling using `MPI_Wtime()` instrumented around every communication primitive and computational phase.
- **Profiling Design:**
- Microsecond timers wrap: `MPI_Scatterv`, Halo exchange (`MPI_Sendrecv` in V5; `MPI_Irecv` / `MPI_Isend` / `MPI_Waitall` in V6), `MPI_Cart_create` / `MPI_Cart_shift` (V6 topology), `MPI_Gatherv`, and `MPI_Barrier`.
- All rank timers are gathered to Rank 0 using `MPI_Gather`.
- Metrics are evaluated on the **critical path rank** (the rank with the maximum overall runtime) to prevent idle wait skews.
- Full matrix evaluated: **2 Versions (V5, V6) × 3 Datasets (Small, Medium, Large) × 3 Rank Counts (2, 8, 16 ranks)** = **18 distinct runs**.
- All 18 configurations were verified against sequential golden baselines (0 mismatches).

B.2.2 Complete MPI Profiling Benchmark Matrix

Below is the verified instrumentation data captured across all 18 configurations:

Version	Dataset	Ranks	Total Time (ms)	Computation (ms)	Communication (ms)	Compute %	Comm %	Dominant MPI Call
Version 5 (Basic)	**Small**	2	3.37	2.49	0.46	73.9%	13.5%	`MPI_Scatterv`
Version 5 (Basic)	**Small**	8	2.92	1.14	1.77	39.3%	60.7%	`MPI_Sendrecv`
Version 5 (Basic)	**Small**	16	5.66	0.31	5.35	5.5%	94.5%	`MPI_Sendrecv`
Version 5 (Basic)	**Medium**	2	14.00	9.96	2.26	71.1%	16.1%	`MPI_Scatterv`
Version 5 (Basic)	**Medium**	8	9.44	5.26	4.18	55.7%	44.3%	`MPI_Sendrecv`
Version 5 (Basic)	**Medium**	16	12.38	1.26	11.12	10.2%	89.8%	`MPI_Sendrecv`
Version 5 (Basic)	**Large**	2	59.73	40.72	9.48	68.2%	15.9%	`MPI_Scatterv`
Version 5 (Basic)	**Large**	8	37.20	10.42	26.78	28.0%	72.0%	`MPI_Gatherv`
Version 5 (Basic)	**Large**	16	34.69	8.34	26.35	24.0%	76.0%	`MPI_Sendrecv`
Version 6 (Optimized)	**Small**	2	6.14	3.42	1.25	55.8%	20.3%	`MPI_Gatherv`
Version 6 (Optimized)	**Small**	8	4.29	1.25	2.10	29.2%	49.0%	`MPI_Scatterv`
Version 6 (Optimized)	**Small**	16	8.03	0.33	5.49	4.1%	68.3%	`MPI_Waitall`
Version 6 (Optimized)	**Medium**	2	16.69	10.31	2.36	61.8%	14.1%	`MPI_Scatterv`
Version 6 (Optimized)	**Medium**	8	12.93	3.27	7.03	25.3%	54.4%	`MPI_Gatherv`
Version 6 (Optimized)	**Medium**	16	11.32	1.23	7.69	10.9%	67.9%	`MPI_Gatherv`
Version 6 (Optimized)	**Large**	2	64.31	40.37	10.02	62.8%	15.6%	`MPI_Gatherv`
Version 6 (Optimized)	**Large**	8	37.37	10.03	22.05	26.8%	59.0%	`MPI_Gatherv`
Version 6 (Optimized)	**Large**	16	37.86	7.03	22.15	18.6%	58.5%	`MPI_Gatherv`

B.2.3 Scaling Analysis (2 vs. 8 vs. 16 Ranks)

Version	Dataset	Speedup (2 → 8)	Speedup (2 → 16)	Comm. Growth (2 → 8)	Comm. Growth (2 → 16)	Compute Reduction (2 → 16)
Version 5 (Basic)	Small	1.16×	0.60× (slowdown)	3.88×	11.72×	8.04×
Version 5 (Basic)	Medium	1.48×	1.13×	1.85×	4.92×	7.91×
Version 5 (Basic)	Large	1.61×	1.72×	2.82×	2.78×	4.88×
Version 6 (Optimized)	Small	1.43×	0.76× (slowdown)	1.69×	4.41×	10.37×
Version 6 (Optimized)	Medium	1.29×	1.47×	2.98×	3.26×	8.37×
Version 6 (Optimized)	Large	1.72×	1.70×	2.20×	2.21×	5.74×

B.2.4 Evaluation of Core Profiling Questions

1. What percentage of total time is spent on communication vs. computation?

- **Low Concurrency (2 Ranks):** Computation represents the clear majority across all datasets:
- Small: Computation **~56% – 74%** | Communication **~14% – 20%**
- Medium: Computation **~62% – 71%** | Communication **~14% – 16%**
- Large: Computation **~63% – 68%** | Communication **~15% – 16%**
- **High Concurrency (16 Ranks):** Communication completely reverses the profile and dominates:
- Small: Communication **~68% – 95%** | Computation **~4% – 6%**
- Medium: Communication **~68% – 90%** | Computation **~10% – 11%**
- Large: Communication **~58% – 76%** | Computation **~19% – 24%**

2. Which MPI operation has the largest measured time?

- `MPI_Gatherv` is the single largest communication bottleneck on Large datasets across ranks (22.05 – 26.78 ms). Collecting the full 33.18 MB image back to Rank 0 serializes socket/shared-memory transfers.
- `MPI_Scatterv` dominates initial distribution at 2 ranks on Medium and Small datasets (9.48 ms on Large).
- `MPI_Sendrecv` dominates the blocking halo phase in Version 5 when rank counts scale to 8 and 16 ranks.
- `MPI_Waitall` becomes the primary bottleneck in Version 6 on Small datasets at 16 ranks (5.49 ms comm), as the interior slice is too small to mask halo transit latency.

3. How does communication time scale with process count (2 vs 8 vs 16)?

- Pure computation scales downward strongly (reducing by **4.88× to 10.37×** from 2 to 16 ranks).
- However, communication time scales upward significantly:
- Small: grows by **1.69× – 3.88×** (at 8 ranks) and **4.41× – 11.72×** (at 16 ranks).
- Medium: grows by **1.85× – 2.98×** (at 8 ranks) and **3.26× – 4.92×** (at 16 ranks).
- Large: grows by **2.20× – 2.82×** (at 8 ranks) and **2.21× – 2.78×** (at 16 ranks).
- This sharp growth in collective and point-to-point communication caps scaling gains beyond 8 ranks.

4. At what process count does communication begin to dominate?

- **Communication begins to exceed computation at 8 ranks across all datasets:**
- On Small at 8 ranks, communication reaches **49.0%** (V6) and **60.7%** (V5).
- On Medium at 8 ranks, communication reaches **54.4%** (V6) and **44.3%** (V5).
- On Large at 8 ranks, communication reaches **59.0%** (V6) and **72.0%** (V5).
- By 16 ranks, communication accounts for over 58% to 95% of execution time across all problem sizes.

5. How does problem size affect the communication-to-computation ratio?

- Follows the **surface-to-volume ratio principle**:
- The communication volume of halo exchanges scales with the 1D boundary perimeter (2 × W), while computation scales with image area (H × W).
- Larger problem sizes sustain higher computational intensity per rank. At 16 ranks, Large still devotes **18.6% – 24.0%** of time to computation, whereas Small shrinks to just **4.1% – 5.5%**.
- As a result, Large achieves **1.70× – 1.72× speedup** at 16 ranks over 2 ranks, whereas Small suffers a significant **slowdown (0.60× – 0.76×)** due to communication dominance.

6. Did Cartesian topology reduce overhead?

- In Version 6, Cartesian topology was established via `MPI_Cart_create` and `MPI_Cart_shift`.
- The setup time was explicitly measured (`CartSetupTime_ms`) and was consistently minimal (**~0.38 – 0.45 ms**), confirming that Cartesian topology provides a clean, portable coordinate abstraction without measurable runtime penalty.

7. Did non-blocking halo exchange overlap communication with computation?

- Version 6 issues non-blocking requests (`MPI_Irecv` / `MPI_Isend`), computes interior rows (rows 2 to local_rows - 1), calls `MPI_Waitall` , and then processes boundary rows.
- **On Medium and Large datasets at 2 ranks**, partial overlap was successful, with `MPI_Waitall` times remaining negligible (<0.05 ms).
- **At 16 ranks on Small datasets**, the number of local interior rows per rank is too small (1080 / 16 ≈ 67 rows). The interior computation finishes before halo packets traverse the MPI transport, resulting in rank stalls at `MPI_Waitall` .
- **Root-to-worker volume vs. Halo volume:** In total bytes, `MPI_Scatterv` and `MPI_Gatherv` transfer 2 × W × H bytes (66.36 MB on Large), while halo exchange moves only 7 × 2 × 7680 = 107.5 KB (only **0.16%** of total communication bytes). Therefore, overlapping halo exchanges can only optimize a small fraction of the program's overall communication footprint.

B.3 Energy Analysis

B.3.1 CPU Energy - Hardware Fallback Statement

Genuine attempt to use RAPL:

```
$ perf stat -e power/energy-pkg/,power/energy-ram/ ./v5_mpi_basic.exe
'perf' is not recognized as an internal or external command,
operable program or batch file.
```

Why it failed: The MPI and OpenMP CPU code was developed and benchmarked on a **Windows host environment** (MS-MPI / MinGW GCC). Intel RAPL counters are exposed via Linux kernel subsystems (`perf` or

`/sys/class/powercap/`), which are absent on native Windows. MS-MPI does not provide software energy measurement APIs.

Fallback applied (per Section B.4): Per the assignment specification - "*If RAPL (CPU energy) is unavailable, report wall-clock time only for CPU-only versions and state in the report that CPU energy could not be measured on the available hardware*" - wall-clock execution times are reported as the primary performance metric for all CPU versions.

B.3.2 GPU Energy Measurement (CUDA Version on NVIDIA Tesla T4)

Unlike the CPU environment, GPU energy was **directly measured via hardware power polling** using `nvidia-smi` on Google Colab (as implemented in `CUDA.ipynb` Cells 17–19).

Measurement Methodology:

1. **Background Power Sampler:** A Python thread actively queried `nvidia-smi --query-gpu=power.draw --format=csv,noheader,nounits -lms 100` at **100 ms polling intervals**.
2. **Idle Baseline:** Before starting execution, board power was sampled for 5 seconds while idle to determine baseline power draw ($\mathbf{P_{idle}}$ **= 10.0 W**).
3. **Sustained Execution Loop:** The dedicated benchmark program `cuda_energy.cu` ran full end-to-end iterations (Host-to-Device transfer → Kernel execution → Device-to-Host transfer) in a continuous loop for **10.092 seconds**, executing **580 complete iterations** on the Large dataset (7680 × 4320, 33.18 MB).
4. **Active Power Sampling:** Board power during sustained load was averaged between t_0 + 1.0s and t_1 - 0.3s to exclude initialization/teardown transients ($\mathbf{P_{active}}$ **= 37.0 W**).

Measured Energy Results:

Metric	Measured Value	Formula / Details
GPU Model	NVIDIA Tesla T4 (Turing)	16 GB GDDR6, 70W TDP
Idle Board Power (P_{idle})	**10.0 W**	5-second pre-execution baseline
Active Board Power (P_{active})	**37.0 W**	Average during sustained execution
Total Test Duration	**10.0916 s**	Continuous execution loop
Iterations Completed	**580**	Full end-to-end runs
Average Wall-Clock Time per Run (t)	**17.399 ms** (0.0174 s)	10.0916 s / 580 iterations
Total Energy per Run (E_{total})	**0.6431 Joules**	$P_{active} \times t$ = 37.0 W × 0.017399 s
Dynamic Energy per Run ($E_{dynamic}$)	**0.4698 Joules**	$(P_{active} - P_{idle}) \times t$ = (37.0 − 10.0) W × 0.017399 s
Estimated Kernel-Only Energy	**~0.0574 Joules**	$P_{active} \times t_{kernel}$ = 37.0 W × 0.00155 s

Energy Efficiency Analysis:

- Processing a massive 33.18-megapixel 4K/8K image on the GPU consumes just **0.643 Joules** of total board energy, and only **~0.057 Joules** during the active compute kernel.
- The GPU draws an average of **37.0 W** under sustained load (roughly 53% of the T4's 70 W TDP cap).
- Despite PCIe data transfers accounting for 87% of the run time, the sheer computational throughput of 2560 CUDA cores allows the workload to finish in 17.4 ms, drastically minimizing the integrated energy-delay product (EDP).

B.3.3 Wall-Clock Time Summary (Across All Implementations)

Version	Small (ms)	Medium (ms)	Large (ms)	Energy (J) [Large]
Sequential	65.92	163.86	629.79	N/A (RAPL unavailable)
OpenMP (2T)	39.42	150.53	424.60	N/A (RAPL unavailable)
Optimized OpenMP (2T)	20.75	23.14	77.27	N/A (RAPL unavailable)
CUDA (Kernel)	0.21	0.42	1.43 – 1.55	**~0.057 J** (est.)
CUDA (Total)	1.42	4.35	15.87 – 17.40	**0.6431 J** (measured)
MPI Basic v5 (4 ranks)	5.805	14.430	57.990	N/A (RAPL unavailable)
MPI Optimized v6 (4 ranks)	6.065	16.432	61.384	N/A (RAPL unavailable)

B.4 Tool Fallback Summary

As required by the assignment guidelines ("*If a tool is not supported on your device*"), the table below documents every required profiling tool, whether it was supported on the test environment, the genuine attempt made, and the verified fallback employed:

Tool	Requirement	Attempted On	Outcome / Reason	Fallback Used	Evidence in Notebooks
`perf` / `gprof`	B.1 CPU Profiling	Google Colab	Virtualized KVM guest disables hardware PMU counters; multi-threaded gprof limited by `ITIMER_PROF`.	`gprof`, `perf record -e cpu-clock`, and Cachegrind	Captured 100% compute hotspot in Sequential (1.13B insts, 0.7% D1 miss), 80.21% in OpenMP, 73.02% in Opt OMP (42 AVX2 insts).
Nsight Compute (`ncu`)	B.1 GPU Profiling	Google Colab (Tesla T4)	`nvprof` returned `command not found` (deprecated on Turing CC 7.5). Ran `ncu --set full`.	`ncu` and `nsys` CLIs	Complete microarchitectural metrics captured: 76.35% occupancy, 49.82% SM throughput, 34% LG queue stall.
`mpiP`	B.2 MPI Profiling	Windows Host (MS-MPI)	`mpiP` requires POSIX timers, libunwind, and binutils; unavailable on Windows.	Manual `MPI_Wtime()` instrumentation	18 configurations profiled across 2, 8, 16 ranks; detailed compute vs. comm split on critical path rank.
Intel RAPL	B.3 CPU Energy	Windows Host & Colab VM	Windows lacks Linux `/sys/class/powercap/` interface; Colab VM does not pass through MSR registers.	Wall-clock execution time reported	CPU energy declared unmeasurable per assignment spec; wall-clock times used as performance proxy.
`nvidia-smi`	B.3 GPU Energy	Google Colab (Tesla T4)	Supported and executed successfully.	Direct hardware power sampling	Background polling at 100 ms intervals over 580 iterations yielded 10.0 W idle, 37.0 W active, 0.6431 J per run.

Part C - Final Consolidated Comparison (All Versions, All Datasets)

Consolidated Performance Table

The table below uses the **best-performing process/thread count** for each parallel version. For OpenMP versions, p=2 (the maximum on Colab's 2-core VM). For MPI versions, the best result across 2/4/8 ranks is selected per dataset.

Version	Small Time (ms)	Medium Time (ms)	Large Time (ms)	Speedup vs Seq. (Large)	Parallel Efficiency	Energy (J)	Top Bottleneck	Directly Comparable To
Sequential	65.92	163.86	629.79	1.00×	-	N/A (RAPL unavailable)	Single-threaded computation	OpenMP, Opt. OpenMP, MPI
OpenMP (2T)	39.42	150.53	424.60	1.48×	74.0% (p=2)	N/A (RAPL unavailable)	Hyperthreading contention, cache pressure	Sequential, Opt. OpenMP
CUDA (Kernel)	0.21	0.42	1.43 – 1.55	406.3× – 440.4×	N/A (2560 cores)	~0.057 J (kernel est.)	L1 queue stalls (uncoalesced loads)	CUDA Total only
CUDA (Total)	1.42	4.35	15.87 – 17.40	36.2× – 39.7×	N/A	**0.6431 J** (measured)	PCIe transfer (87.0% of total time)	CPU versions (end-to-end)
Optimized OpenMP (2T)	20.75	23.14	77.27	8.15×	407.5% (superlinear)	N/A (RAPL unavailable)	SIMD saturation at small sizes	Sequential, OpenMP
MPI Basic (best)	5.805 (4R)	14.430 (4R)	57.249 (8R)	11.00×	1.38 (p=8)	N/A (RAPL unavailable)	Process overhead at high rank counts	MPI Optimized, OpenMP
MPI Optimized (best)	5.322 (2R)	16.432 (4R)	61.384 (4R)	10.26×	2.57 (p=4)	N/A (RAPL unavailable)	Loop-splitting overhead, shared-memory latency masking	MPI Basic

Note: "2T" = 2 threads, "4R" = 4 ranks, "8R" = 8 ranks. Parallel efficiency for optimized OpenMP exceeds 100% due to algorithmic optimizations (branch elimination, SIMD) that improve per-thread performance beyond the sequential baseline.

Comparability Discussion

Same-Resource Comparisons (CPU-only versions)

Sequential vs. OpenMP vs. Optimized OpenMP vs. MPI all execute on CPU hardware and are directly comparable on wall-clock time and speedup:

- The **MPI Basic (v5)** version at 4–8 ranks significantly outperforms both OpenMP versions despite running on the same physical machine, because MPI spawns separate processes that each get their own address space, reducing cache contention. MPI with 4 ranks on the Large dataset (57.99 ms) is **10.9× faster** than sequential and **7.3× faster** than basic OpenMP.

- The **Optimized OpenMP** version (77.27 ms on Large) slightly underperforms MPI at 4–8 ranks, but achieves this with only 2 threads through algorithmic optimization (SIMD, branch elimination).
- **Sequential** is the clear baseline at 629.79 ms on Large.

Cross-Hardware Comparisons (GPU vs. CPU)

CUDA vs. CPU versions are **not directly comparable** on raw time:

- The CUDA **kernel-only** speedup (492× for Large) reflects pure GPU parallelism across 2560 CUDA cores. This is not a fair comparison to 2-thread OpenMP.
- The CUDA **total end-to-end** speedup (37.89× for Large) is more comparable, as it includes the PCIe data transfer cost that is analogous to file I/O overhead in CPU versions.
- The CUDA total time (16.62 ms for Large) is still **3.4× faster** than the best MPI result (57.25 ms), demonstrating that GPU parallelism is beneficial even after accounting for transfer overhead.

Scaling Comparisons (MPI vs. OpenMP)

MPI process count vs. OpenMP thread count are not equivalent resource units:

- OpenMP threads share memory and cache hierarchy; adding threads on a single physical core (hyperthreading) yields <2× speedup.
- MPI ranks are separate processes with independent address spaces; even on a shared-memory machine, the MPI runtime can exploit separate cache lines and avoid false sharing.
- **Parallel efficiency comparison:** OpenMP at p=2 achieves 74% efficiency on Large, while MPI Basic at p=4 achieves 2.72× efficiency per rank (sequential baseline / (p × T_mpi) = 629.79 / (4 × 57.99) = 2.72). This apparent super-linear efficiency is an artifact of the single-core Colab baseline - the MPI processes run on a different machine with different core counts and clock speeds.

Energy Comparisons

CPU-only and GPU versions draw power from different components:

- **CPU energy (package + RAM):** Evaluated via Intel RAPL - **unavailable on Windows host hardware** (MS-MPI / MinGW). Wall-clock execution time is reported as the proxy metric per Section B.4.
- **GPU energy (board power):** Directly measured on NVIDIA Tesla T4 via continuous background `nvidia-smi` sampling at 100 ms intervals across 580 iterations on the Large dataset.
- Baseline Idle Board Power: **10.0 W**
- Active Board Power: **37.0 W**
- End-to-End Energy per Run: **0.6431 Joules** (Dynamic: **0.4698 Joules**)
- Isolated Kernel Energy Estimate: **~0.0574 Joules**

Conclusion

Fastest Version

CUDA (kernel-only) is the fastest implementation, completing the Large dataset in **1.43 – 1.55 ms** (406× – 440× speedup over sequential baseline). Even when including host-to-device and device-to-host PCIe transfer overhead (which constitutes 87% of execution time), CUDA total end-to-end time is **15.87 – 17.40 ms** (36.2× – 39.7× speedup), remaining significantly faster than any CPU implementation.

Among CPU-only versions, **MPI Basic at 8 ranks** achieves the best wall-clock time on the Large dataset (**57.25 ms**, 11.00× speedup), followed closely by **MPI Basic at 4 ranks** (**57.99 ms**, 10.86× speedup).

Most Energy-Efficient Version

- **CUDA on Tesla T4** demonstrated outstanding absolute energy efficiency, requiring only **0.6431 Joules** per end-to-end execution on the massive 33.18 MP Large dataset (and only **~0.057 Joules** for the compute kernel alone).
- While CPU energy in Joules could not be directly measured on the Windows host due to RAPL unavailability, **Optimized OpenMP** achieves the best energy-performance balance among CPU solutions: delivering an **8.15× speedup** using only 2 threads (77.27 ms on Large) by virtue of SIMD vectorization and branch elimination, avoiding the process and scheduling power overhead of 8 MPI ranks.

Best Parallel Efficiency

- **Optimized OpenMP** exhibits superlinear speedup (8.15× on 2 threads → >400% efficiency) due to algorithmic improvements that go beyond mere parallelization.
- **MPI Basic** at p=4 achieves good scaling (10.86× on Large), though efficiency decreases at p=8 due to communication overhead on shared memory.

Fastest vs. Most Resource-Efficient

The fastest version (CUDA) is **not** the most resource-efficient when considering total hardware cost: - CUDA requires an expensive discrete GPU (Tesla T4) consuming significantly more power than the CPU alone. - The 492× kernel speedup drops to 37.89× when PCIe transfer is included - a **13× penalty** from data movement. - The **Optimized OpenMP** version achieves a remarkable balance: 8.15× speedup using only 2 CPU threads, with zero additional hardware cost and minimal power overhead above the sequential baseline.

This demonstrates the principle that **algorithmic optimization and hardware-aware coding (SIMD, branch elimination, cache optimization) can be more cost-effective than simply adding more parallel resources**.

Individual Contributions (Programming Assignment 2)

Group Member	Roll Number	Role & Contribution
Adwaith J	2024BCD0018	**Part B.3 (Energy Analysis) & Part C (Consolidated Comparison):** Conducted energy analysis and hardware power fallback evaluation (RAPL vs. `nvidia-smi`), assembled the final consolidated performance comparison matrix across all six implementations, and authored the cross-hardware comparability and efficiency discussions.
Noel Manoj (Group Leader)	2024BCS0046	**Part B.1 (CPU & GPU Profiling) & Part B.2 (MPI Profiling):** Executed microarchitectural profiling across CPU versions (`gprof`, `perf record`, Valgrind Cachegrind) and GPU (Nsight Systems & Nsight Compute), instrumented granular `MPI_Wtime()` profiling on critical-path ranks for Version 5 and Version 6, and conducted communication-to-computation scaling analysis.
Aditya A	2024BCS0038	**Part A (MPI Implementations) & Report Drafting:** Implemented Version 5 (Basic MPI with 1D domain decomposition, Scatterv/Gatherv, and blocking halo exchange) and Version 6 (Optimized MPI with 1D Cartesian topology and non-blocking halo overlap), validated numerical verification against golden references, and led overall report drafting and documentation.